PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

Facultad de Ingeniería

Escuela de Ingeniería Informática

INFORME TRABAJO FINAL

INF3541-2 TALLER DE BASE DE DATOS

Integrantes: *Agustín Guzmán y Nicolás Leiva*

Fecha: *noviembre de 2026*

---

1. Introducción

Este informe documenta el diseño, la carga y la explotación de una base de datos PostgreSQL construida a partir de un catálogo de películas. Se describen el modelo, la carga mediante tablas de traspaso, los triggers de auditoría, los procedimientos almacenados, las consultas comparadas y el control de acceso.

2. Descripción del dataset

2.1. Fuente y autorización

El dataset es un catálogo de películas compuesto por diez archivos CSV, publicado originalmente en 2024. No cumple estrictamente el requisito de recencia del enunciado, pero su uso fue autorizado por el profesor.

Fuente: https://www.kaggle.com/datasets/gsimonx37/letterboxd.

3. Modelo de datos

3.1. Diagrama entidad-relación

El modelo tiene 19 tablas y está en tercera forma normal. La Figura 1 muestra el diagrama.

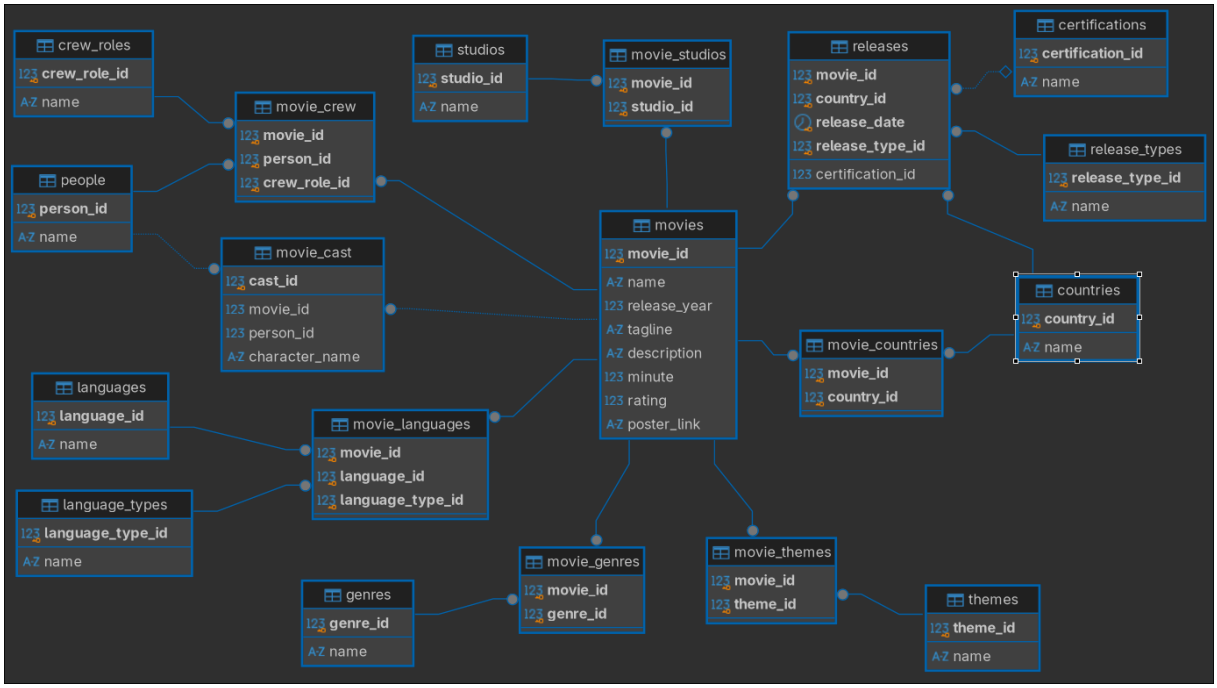


Figura 1: Diagrama entidad-relación del modelo.

4. Integridad y reglas de negocio

El esquema usa claves primarias, claves foráneas, restricciones de unicidad y restricciones `CHECK`. Las principales validan nombre no vacío, año entre 1870 y 2100, duración positiva, rating entre 0 y 5, fechas entre 1870 y 2100 y póster nulo o no vacío.

Las claves foráneas de las relaciones usan `ON DELETE CASCADE` hacia `movies`. Las claves hacia catálogos y personas impiden borrar un registro que esté en uso. Las reglas que involucran varias tablas se controlan en los procedimientos almacenados.

5. Carga de datos

Los CSV se copian con `\copy` a tablas `UNLOGGED` del schema `staging`, sin claves foráneas ni restricciones. Luego los scripts de `load` traspasan los datos aplicando `CAST`, `NULLIF`, `DISTINCT` y `JOIN` para resolver los catálogos por nombre.

6. Triggers de auditoría

La tabla `audit_log` registra la tabla afectada, la operación (`I`, `U` o `D`), la clave de la fila, los datos anteriores y nuevos en `JSONB`, el usuario y la fecha. Una función genérica es reutilizada por cuatro triggers `AFTER INSERT OR UPDATE OR DELETE` sobre `movies`, `people`, `studios` y `releases`.

Los triggers se crean después de la carga para no registrar las inserciones masivas. Una prueba de alta, modificación y baja deja este log:

```
 operation | row_key
-----------+------------------------
 I         | {"movie_id": 999999999}
 U         | {"movie_id": 999999999}
 D         | {"movie_id": 999999999}
```

7. Procedimientos almacenados

Se implementaron cinco procedimientos para los mantenedores:

Procedimiento	comportamiento
`sp_movie_upsert`	Exige nombre no vacío, año entre 1870 y 2100, duración positiva y rating entre 0 y 5.
`sp_movie_delete`	Eliminación de una película. Las relaciones se eliminan por `ON DELETE CASCADE`.
`sp_studio_delete`	No permite eliminar un estudio con películas asociadas.
`sp_person_merge`	Fusiona dos personas: mueve reparto y crew y elimina el duplicado.
`sp_register_release`	Registra un estreno. La fecha no puede ser anterior al año de la película y crea los catálogos que falten.

Tabla 1: Procedimientos almacenados y sus reglas de negocio.

Todos validan sus datos y terminan con `COMMIT`. Si se incumple una regla de negocio hacen `ROLLBACK` y lanzan una excepción; por ejemplo, `sp_studio_delete` rechaza el borrado si el estudio tiene películas. Los procedimientos no usan `SECURITY DEFINER`, porque PostgreSQL no permite `COMMIT` ni `ROLLBACK` en ellos; por eso el rol `bd_mantenedor` tiene permisos `DML` sobre las tablas que mantiene.

8. Consultas y optimización

8.1. Consultas equivalentes

La consulta A obtiene películas de Drama o Comedia, estrenadas en Estados Unidos entre 2000 y 2009, con rating mayor o igual a 4.0. Usa enlace por `WHERE`, `OR` e `IN`, sin `JOIN`. La consulta B devuelve los mismos resultados con `JOIN`, una CTE y `EXISTS`. Ambas consultas devuelven 71 películas.

8.2. Planes antes de la optimización

Ambas consultas devuelven 71 películas. Con los índices base, el planificador usa dos índices de una columna y luego visita el heap:

```
Consulta A (WHERE, OR, IN)
Unique (actual rows=71)
  -> Bitmap Heap Scan on movies (actual rows=296)
       -> BitmapAnd
            -> Bitmap Index Scan on idx_movies_rating
            -> Bitmap Index Scan on idx_movies_release_year
Execution Time: 10.163 ms

Consulta B (JOIN, CTE, EXISTS)
Unique (actual rows=71)
  -> Nested Loop Semi Join
       -> Bitmap Heap Scan on movies (actual rows=296)
            -> BitmapAnd
                 -> Bitmap Index Scan on idx_movies_rating
                 -> Bitmap Index Scan on idx_movies_release_year
Execution Time: 5.891 ms
```

8.3. Índices creados

Se creó un índice que cubre el filtro de año y rating y devuelve las columnas del `SELECT` sin visitar el heap:

```
CREATE INDEX idx_movies_year_rating
    ON movies (release_year, rating) INCLUDE (movie_id, name);
```

También se probaron índices compuestos por género y por país, pero el planificador partía del catálogo y recorría cientos de miles de filas, por lo que resultaban más lentos. Se descartaron y se eliminaron los índices de una sola columna, porque las claves primarias de las relaciones ya cubren la búsqueda por `movie_id`.

8.4. Planes después de la optimización

Los dos planes cambian el acceso a `movies` por un `Index Only Scan` sobre el nuevo índice, sin visitar el heap:

```
Consulta A
Unique (actual rows=71)
  -> Index Only Scan using idx_movies_year_rating on movies
       (actual rows=296)
Execution Time: 1.766 ms

Consulta B
Unique (actual rows=71)
  -> Nested Loop Semi Join
       -> Index Only Scan using idx_movies_year_rating on movies
            (actual rows=296)
Execution Time: 4.287 ms
```

8.5. Conclusión de la optimización

El costo estimado bajó de 3798 a 1236 en la consulta A y de 3837 a 1278 en la B. En la práctica, A pasó de 10.163 ms a 1.766 ms y B de 5.891 ms a 4.287 ms, con caché caliente y los mismos 71 resultados.

La mejora se debe a que el filtro de año y rating se resuelve con un `Index Only Scan` que ya devuelve `movie_id` y `name`. Los índices por género y país se descartaron porque inducían un plan que partía del catálogo y recorría 174.489 filas de `movie_countries`, más lento que partir de las 296 películas filtradas.

9. Usuarios y vistas

Se crearon cinco vistas con filtros: `vw_movies_public` oculta descripción y póster; `vw_movies_recent` muestra solo películas desde 2000; `vw_releases_public` muestra solo estrenos ya ocurridos; `vw_releases_upcoming` reserva los estrenos futuros; y `vw_movie_cast_public` muestra el reparto con nombres legibles.

La Tabla 2 resume los niveles de acceso.

Rol	Acceso
`bd_lectura`	Solo las vistas públicas. No puede leer las tablas base.
`bd_mantenedor`	`SELECT`, `DML` sobre las tablas mantenidas y ejecución de los procedimientos.
`bd_admin`	Control total del esquema, tablas, secuencias, procedimientos y `audit_log`.

Tabla 2: Roles y niveles de acceso.

Con `SET ROLE` se comprobó que cada rol accede solo a lo permitido y que `audit_log` rechaza `UPDATE` y `TRUNCATE`.

10. Conclusiones

- La carga en dos capas permitió separar los datos crudos de las transformaciones y validar los conteos al final.
- El modelo de 19 tablas evita redundancia y mantiene la integridad mediante claves y restricciones.
- La comparación de planes permite justificar los índices creados y su efecto en las consultas.
- Los roles y las vistas permiten exponer el catálogo sin dar acceso directo a las tablas base.